

前言

刷题推荐: [HDLBits](#)

🗨️ 第一章：Verilog 基础入门

1.1 数字逻辑基础

📖 概念讲解

在数字电路设计中，**逻辑门**和**触发器**是实现电路功能的基本单元。在 Verilog 中，我们通过代码描述硬件行为，而在 C 语言中，类似的行为可以通过逻辑运算符来模拟。但 Verilog 的独特之处在于，它直接面向硬件设计。

实际应用：

逻辑门的应用非常广泛，例如，在计算机的算术逻辑单元（ALU）中，AND 和 OR 门是实现算术运算、条件判断等功能的核心基础。

代码示例：从简单逻辑门到实际电路

以下代码展示了如何在 Verilog 中实现基础的逻辑门操作。我们通过简单的代码实现了 **与门 (AND)**、**或门 (OR)** 等基本逻辑运算。随后，我们将结合这些逻辑门的 **真值表**，展示它们的工作原理。

Verilog 实现基本逻辑门

```
1 module logic_gate(input a, input b, output and_out, output or_out);
2     assign and_out = a & b; // 实现与门 (AND)
3     assign or_out = a | b; // 实现或门 (OR)
4 endmodule
```

Fence 1

逻辑门及其真值表

输入 A	输入 B	与门 (A & B)	或门 (A B)	非门 (~A)	异或门 (A ^ B)
0	0	0	0	1	0
1	0	0	1	0	1
0	1	0	1	1	1
1	1	1	1	0	0

Table 1

在以上真值表中：

- 🔴 **与门 (AND)**：仅当 A 和 B 都为 1 时，输出为 1。
- 🔴 **或门 (OR)**：只要 A 或 B 为 1，输出就为 1。

- **非门 (NOT)**: 将输入的值取反。
- **异或门 (XOR)**: 当且仅当 A 和 B 不相同, 输出为 1。

通过简单的 Verilog 代码和真值表的结合, 可以清晰地理解逻辑门的基本功能及其工作原理。

实际案例：门禁系统的基本逻辑

背景: 假设我们要设计一个简单的门禁系统, 只有当两个条件同时满足时 (例如指纹匹配 AND 密码正确), 门才能打开。

门禁系统逻辑门描述 (Verilog 代码)

```
1  module access_control(  
2      input fingerprint_valid, // 指纹验证结果  
3      input password_correct,  // 密码验证结果  
4      output unlock_door       // 门锁信号  
5  );  
6      assign unlock_door = fingerprint_valid & password_correct; // 只有两者都满足时开  
7  endmodule
```

Fence 2

测试代码

```
1  module testbench;  
2      reg fingerprint_valid, password_correct;  
3      wire unlock_door;  
4  
5      access_control uut (  
6          .fingerprint_valid(fingerprint_valid),  
7          .password_correct(password_correct),  
8          .unlock_door(unlock_door)  
9      );  
10  
11     initial begin  
12         // 测试用例 1: 指纹和密码均正确  
13         fingerprint_valid = 1;  
14         password_correct = 1;  
15         #10;  
16         $display("Unlock Door: %b", unlock_door); // 输出应为 1  
17  
18         // 测试用例 2: 仅指纹正确  
19         fingerprint_valid = 1;  
20         password_correct = 0;  
21         #10;  
22         $display("Unlock Door: %b", unlock_door); // 输出应为 0  
23  
24         // 测试用例 3: 指纹和密码均错误  
25         fingerprint_valid = 0;  
26         password_correct = 0;  
27         #10;  
28         $display("Unlock Door: %b", unlock_door); // 输出应为 0  
29     end
```

30 | endmodule

Fence 3

1.2 Testbench 使用

📖 什么是 Testbench?

Testbench 是用于验证设计模块 (DUT, Design Under Test) 是否按照预期工作的测试平台。它通过驱动输入信号、监控输出信号、生成波形文件等功能, 帮助设计人员验证模块的正确性。**Testbench 不会被综合成硬件, 只在仿真工具中运行。**

📖 Testbench 的基本结构

一个标准的 Verilog Testbench 通常由以下几个部分组成:

1. **模块声明**: 定义 Testbench 的名称。
2. **信号声明**: 为 DUT 提供输入信号, 并接收 DUT 的输出信号。
3. **实例化 DUT**: 将设计模块实例化, 并将信号连接到 DUT 的端口。
4. **激励生成**: 控制输入信号的变化, 模拟模块的运行环境。
5. **监视输出**: 实时查看 DUT 的输出信号, 验证其行为是否正确。
6. **生成波形文件 (可选)**: 记录信号的变化, 便于波形工具分析。

📖 Testbench 编写步骤

1. 模块声明

Testbench 是一个 Verilog 模块, 但与 DUT 不同, Testbench 没有输入或输出端口。以下是 Testbench 模块的声明示例:

```
1 | module tb_and_gate;
```

Fence 4

2. 信号声明

Testbench 中的信号用于与 DUT 通信, 包括输入信号和输出信号:

- **reg 类型**: 用于驱动 DUT 的输入信号, 可以在 `initial` 或 `always` 块中赋值。
- **wire 类型**: 用于接收 DUT 的输出信号, 由 DUT 驱动。

例如:

```

1 |   reg a;      // DUT 的输入信号 a
2 |   reg b;      // DUT 的输入信号 b
3 |   wire y;     // DUT 的输出信号 y

```

Fence 5

3. 实例化 DUT

在 Testbench 中需要实例化设计模块 (DUT)，并将 Testbench 的信号连接到 DUT 的端口。实例化方式如下：

```

1 |   and_gate uut ( // 实例化 DUT, 名称为 "uut" (unit under test)
2 |       .a(a),    // DUT 的输入端口 a 连接 Testbench 的信号 a
3 |       .b(b),    // DUT 的输入端口 b 连接 Testbench 的信号 b
4 |       .y(y)     // DUT 的输出端口 y 连接 Testbench 的信号 y
5 |   );

```

Fence 6

4. 激励生成

Testbench 的核心功能是通过输入信号驱动 DUT。使用 `initial` 块生成信号激励 (stimulus)，测试不同的输入组合。

激励生成示例如下：

```

1 |   initial begin
2 |       a = 0; b = 0; // 输入 00
3 |       #10;          // 等待 10 个时间单位
4 |       a = 0; b = 1; // 输入 01
5 |       #10;          // 等待 10 个时间单位
6 |       a = 1; b = 0; // 输入 10
7 |       #10;          // 等待 10 个时间单位
8 |       a = 1; b = 1; // 输入 11
9 |       #10;          // 等待 10 个时间单位
10 |
11 |       $stop;        // 结束仿真
12 |   end

```

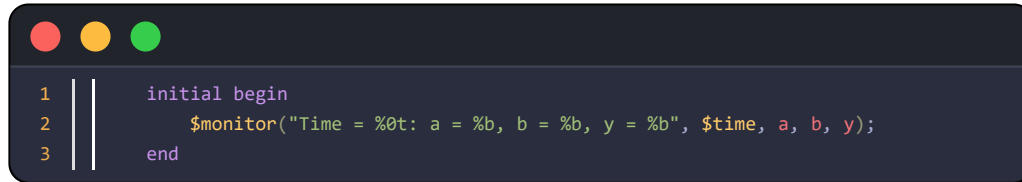
Fence 7

详细解释：

- **initial 块**：一次性执行的代码块，用于设置初始值和生成激励。
- **#10**：延迟 10 个时间单位。
- **\$stop**：停止仿真，通常在激励结束后调用。

5. 监视输出

为了验证 DUT 的输出是否正确，可以使用 `$monitor` 或 `$display` 打印信号的值：



```

1 | initial begin
2 |     $monitor("Time = %0t: a = %b, b = %b, y = %b", $time, a, b, y);
3 | end
  
```

Fence 8

详细解释：

- `$monitor`：实时打印信号的值，每当信号发生变化时输出结果。
- `$time`：系统函数，返回当前仿真时间。
- `%b`：以二进制格式打印信号值。

6. 生成波形文件

在仿真中生成 VCD (Value Change Dump) 文件，用于查看波形。添加 `$dumpfile` 和 `$dumpvars` 任务即可生成波形文件：



```

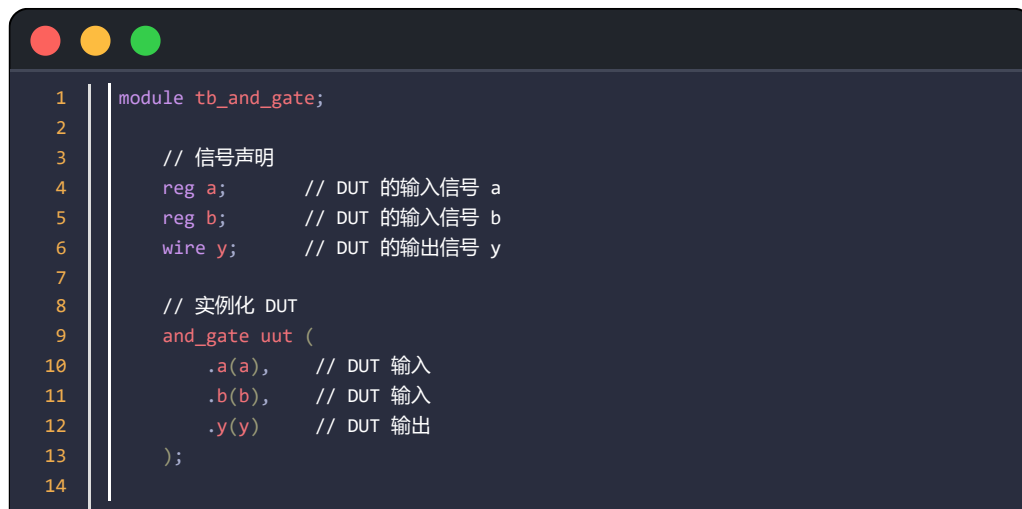
1 | initial begin
2 |     $dumpfile("tb_and_gate.vcd"); // 指定 VCD 文件的名称
3 |     $dumpvars(0, tb_and_gate);    // Dump 当前模块的所有信号
4 | end
  
```

Fence 9

- `$dumpfile`：指定生成的 VCD 文件名称。
- `$dumpvars`：记录指定模块及其子模块的所有信号变化。

完整的 Testbench 示例

以下是完整的 Testbench 示例代码：



```

1 | module tb_and_gate;
2 |
3 |     // 信号声明
4 |     reg a;      // DUT 的输入信号 a
5 |     reg b;      // DUT 的输入信号 b
6 |     wire y;     // DUT 的输出信号 y
7 |
8 |     // 实例化 DUT
9 |     and_gate uut (
10 |         .a(a),    // DUT 输入
11 |         .b(b),    // DUT 输入
12 |         .y(y)     // DUT 输出
13 |     );
14 |
  
```

```

15 // 激励生成
16 initial begin
17     a = 0; b = 0; // 输入 00
18     #10;
19     a = 0; b = 1; // 输入 01
20     #10;
21     a = 1; b = 0; // 输入 10
22     #10;
23     a = 1; b = 1; // 输入 11
24     #10;
25
26     $stop; // 结束仿真
27 end
28
29 // 输出监视
30 initial begin
31     $monitor("Time = %0t: a = %b, b = %b, y = %b", $time, a, b, y);
32 end
33
34 // 生成波形文件
35 initial begin
36     $dumpfile("tb_and_gate.vcd"); // 指定 VCD 文件名称
37     $dumpvars(0, tb_and_gate); // Dump 当前模块的所有信号
38 end
39
40 endmodule

```

Fence 10

仿真流程

1. 编译 DUT 和 Testbench:

使用仿真工具（如 Icarus Verilog）编译 Verilog 文件：

```
1 | iverilog -o tb_and_gate tb_and_gate.v and_gate.v
```

Fence 11

2. 运行仿真:

执行生成的仿真文件，生成 VCD 文件：

```
1 | vvp tb_and_gate
```

Fence 12

3. 查看波形:

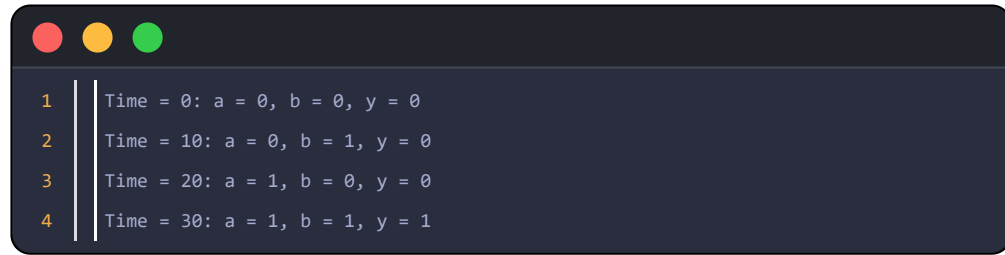
使用波形工具（如 GTKWave）查看生成的波形文件：

```
1 | gtkwave tb_and_gate.vcd
```

Fence 13

📖 仿真结果分析

- 仿真终端会输出信号的值变化，例如：

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It displays four lines of simulation output, each preceded by a line number (1-4) in a light blue font. The output shows the values of signals a, b, and y at different time intervals.

```
1 | Time = 0: a = 0, b = 0, y = 0
2 | Time = 10: a = 0, b = 1, y = 0
3 | Time = 20: a = 1, b = 0, y = 0
4 | Time = 30: a = 1, b = 1, y = 1
```

Fence 14

- 打开波形文件，你可以直观地看到信号的时间序列变化，验证 DUT 的功能是否正确。

📖 总结

- Testbench 的核心作用**是通过输入激励验证设计模块的行为。
 - 组成部分**包括信号声明、实例化 DUT、激励生成、输出监视和波形文件生成。
 - 仿真流程**需要使用工具（如 Icarus Verilog）编译、运行并生成波形文件。
-

☺ 第二章：数据类型基础和语法细节

2.1 数据类型： `reg` 和 `wire`

在 Verilog 中，`reg` 和 `wire` 是两种最常用的数据类型。了解它们的区别和使用场景是 Verilog 编程的核心基础。通过实际案例，你可以清晰地掌握如何在组合逻辑和时序逻辑中选择合适的数据类型。

🔌 `wire`：信号连接的导线

定义与特性

- 🔴 `wire` 的本质：组合逻辑中的信号连接，用于实时传递值，无法存储信号。
- 🔴 特性：
 - 🔴 只能通过连续赋值（`assign`）语句或模块的输出端口驱动。
 - 🔴 `wire` 的值会根据驱动源的变化实时更新，表示信号连接的“导线”。
- 🔴 使用场景：
 - 🔴 组合逻辑电路中传递信号。
 - 🔴 连接模块输入、输出端口的信号。

实际案例：LED 灯的开关控制

假设你设计一个简单的 LED 控制电路，有两个输入信号 `a` 和 `b`，LED 只有当两者都为 1 时点亮。

```
1 | module led_control(input a, input b, output led);
2 |     assign led = a & b; // 组合逻辑: a 和 b 同时为 1 时, led 输出 1
3 | endmodule
```

Fence 15

扩展应用：多条件报警系统

在多传感器报警系统中，可以使用 `wire` 实现传感器信号的组合判断：

```
1 | module alarm_system(input smoke, input heat, output alarm);
2 |     assign alarm = smoke | heat; // 任意一个传感器检测到异常, 报警信号激活
3 | endmodule
```

Fence 16

📦 reg：信号的存储单元

定义与特性

○ **reg 的本质：** 存储单元，用于保存信号值，通常用于时序逻辑中。

○ **特性：**

○ 只能在过程块中（`always` 或 `initial`）赋值。

○ 信号的值保持不变，直到下一个触发事件发生。

○ **使用场景：**

○ 时序逻辑（如触发器、寄存器）的建模。

○ 存储在时钟边沿或特定条件下的信号值。

实际案例：计数器设计

一个简单的计数器，在时钟信号的每个上升沿递增计数值：

```
1 | module counter(input clk, output reg [3:0] count);  
2 |     always @(posedge clk) begin  
3 |         count <= count + 1; // 每个时钟周期递增  
4 |     end  
5 | endmodule
```

Fence 17

扩展应用：数据同步

在通信系统中，通常需要对输入信号进行同步，以确保数据稳定：

```
1 | module data_sync(input clk, input in_signal, output reg synced_signal);  
2 |     always @(posedge clk) begin  
3 |         synced_signal <= in_signal; // 在时钟信号下同步输入信号  
4 |     end  
5 | endmodule
```

Fence 18

📦 向量 (Vector)：多位信号表示

定义与特性

○ **向量的本质：** 多位信号，用于表示数据总线或多位数据。

○ **特性：**

○ 可以对多位信号进行操作（如加法、位操作、拼接）。

- 常用于总线信号建模。
- 使用场景：
 - 表示数据总线（如 8 位数据总线、32 位地址总线）。
 - 复杂逻辑电路中的多位数据操作。
- 书写形式：



```
1 | [n-1:0] dat; //表示n位的向量
2 | //在c语言里面，等价成
3 | dat[n-1];
```

实际案例：4 位加法器

Fence 19

使用向量描述两个 4 位数据相加：



```
1 | module adder_4bit(input [3:0] a, input [3:0] b, output [4:0] sum);
2 |     assign sum = a + b; // 多位信号相加
3 | endmodule
```

Fence 20

扩展应用：数据部分选择与拼接

例如，从 8 位数据中提取部分信号并重新组合：



```
1 | module vector_example(input [7:0] data, output [3:0] lower, output [7:0] reversed);
2 |     assign lower = data[3:0]; // 提取低 4 位
3 |     assign reversed = {data[3:0], data[7:4]}; // 拼接：交换高低 4 位
4 | endmodule
```

Fence 21

📦 Memory：存储器的建模

定义与特性

- **Memory 的本质：**存储多个信号的数组，通常用于描述 RAM、ROM。
- **特性：**
 - Memory 是一组向量信号的集合。
 - Memory 使用 `reg` 声明，需要通过索引访问。
- **使用场景：**
 - 寄存器文件、RAM、ROM 等存储结构的建模。

- 仿真中保存大量数据或状态。
- 书写形式：

○

```
1 [n-1:0] dat [m-1]; //表示m个n位的向量
2 //在c语言里面，等价成
3 dat[m-1][n-1];
```

实际案例：RAM 模型

Fence 22

以下代码描述了一个简单的 16x8 RAM，可以读写数据：

```
1 module ram(input clk, input [3:0] addr, input [7:0] data_in, input write_enable,
2 output reg [7:0] data_out);
3     reg [7:0] memory [0:15]; // 定义 16x8 的存储器
4     always @(posedge clk) begin
5         if (write_enable) begin
6             memory[addr] <= data_in; // 写入数据
7         end else begin
8             data_out <= memory[addr]; // 读取数据
9         end
10    end
11 endmodule
```

Fence 23

扩展应用：存储器初始化

在 FPGA 中，初始化存储器用于加载预设数据：

```
1 initial begin
2     memory[0] = 8'hAA;
3     memory[1] = 8'hBB;
4     memory[2] = 8'hCC;
5 end
```

Fence 24

 wire 和 reg 的区别与总结

特性	wire	reg
是否存储值	不存储值	可以存储值
赋值方式	通过 assign 或模块端口驱动	通过 always 或 initial 块赋值
使用场景	组合逻辑的信号连接	时序逻辑或需要存储值的场景

Table 2

特性	wire	reg
赋值时间	实时更新	事件触发时更新

综合案例：计数器与存储器结合

以下代码展示了如何结合 `reg`、`wire`、向量和 Memory 实现复杂功能：

```
1  module comprehensive_example(  
2      input clk,  
3      input [7:0] a,  
4      input [7:0] b,  
5      input [3:0] addr,  
6      input write_enable,  
7      output reg [7:0] sum,  
8      output reg [7:0] mem_out  
9  );  
10     wire [7:0] temp;           // wire 用于传递组合逻辑信号  
11     reg [7:0] memory [0:15];   // 定义一个 16x8 的存储器  
12  
13     assign temp = a + b;        // 组合逻辑: 计算 a 和 b 的和  
14     always @(posedge clk) begin  
15         sum <= temp;           // 存储 temp 值  
16         if (write_enable) begin  
17             memory[addr] <= sum; // 将结果写入存储器  
18         end else begin  
19             mem_out <= memory[addr]; // 从存储器读取值  
20         end  
21     end  
22 endmodule
```

Fence 25

场景分析

- 组合逻辑部分：通过 `wire` 实现加法操作。
- 时序逻辑部分：`reg` 保存计算结果，并控制数据存储。
- Memory 部分：建模存储器以支持读写操作。

小结

通过本节内容，你可以掌握：

- `wire` 和 `reg` 的使用场景与区别。
- 向量的灵活应用与多位信号操作。
- Memory 的建模与初始化方法。
- 结合实际应用设计复杂硬件电路。

2.2 阻塞与非阻塞赋值

Verilog 中的阻塞赋值（`=`）和非阻塞赋值（`<=`）是描述过程赋值的两种关键形式，它们在执行顺序和硬件实现上存在显著差异。正确理解和使用它们是 Verilog 编程的核心技能，尤其是在时序逻辑和组合逻辑设计中。

阻塞赋值 `=`

定义

- 阻塞赋值是按**顺序执行**的赋值方式。
- 每一条阻塞赋值语句执行完毕后，才会继续执行下一条语句。
- 类似于软件编程中逐条执行的命令式代码。

执行特性

- 当前赋值语句的计算和更新立即生效。
- 所有后续语句都会基于当前语句更新的值继续执行。

适用场景

- 描述组合逻辑（如简单的逻辑门或优先级判断）。
- 建模简单的信号传递行为。

实际案例：组合逻辑建模

设计一个 4 位输入信号的优先级编码器，当 `in[3]` 为高时优先输出对应编码。

```

1  module priority_encoder(input [3:0] in, output reg [1:0] out);
2      always @(*) begin
3          if (in[3])
4              out = 2'b11; // 优先级最高
5          else if (in[2])
6              out = 2'b10;
7          else if (in[1])
8              out = 2'b01;
9          else
10             out = 2'b00; // 默认值
11         end
12     endmodule

```

Fence 26

注意事项

- 在时序逻辑中使用阻塞赋值可能导致竞态问题，不推荐用于时钟触发块。

非阻塞赋值 <=

定义

- 非阻塞赋值是按**并行执行**的赋值方式。
- 所有非阻塞赋值语句的右值在同一时间计算，但其更新将在**当前时间步结束后**生效。

执行特性

- 所有赋值语句的右值在当前时间点计算完成。
- 左值更新延迟到当前时间步结束后，同时生效。

适用场景

- 描述时序逻辑（如寄存器或触发器）。
- 建模复杂的时钟驱动电路。

实际案例：寄存器建模

设计一个简单的 D 触发器，在时钟上升沿存储输入信号。

```

1  module d_flip_flop(input clk, input d, output reg q);
2      always @(posedge clk) begin
3          q <= d; // 非阻塞赋值，描述触发器行为
4      end
5  endmodule

```

Fence 27

注意事项

- 非阻塞赋值适用于所有时钟触发逻辑，避免了竞态问题，且易于硬件综合。

阻塞与非阻塞赋值的区别

示例：同时使用阻塞与非阻塞赋值

```

1  module blocking_vs_nonblocking;
2      // 定义变量
3      reg clk;          // 时钟信号
4      reg a_blocking, b_blocking, c_blocking; // 阻塞赋值变量
5      reg a_nonblocking, b_nonblocking, c_nonblocking; // 非阻塞赋值变量
6

```

```

7      // 时钟信号生成
8      initial begin
9          clk = 0;
10         forever #5 clk = ~clk; // 每 5 个时间单位翻转时钟
11     end
12
13     // 测试阻塞赋值
14     always @(posedge clk) begin
15         a_blocking = 1;           // 阻塞赋值: 按顺序执行
16         b_blocking = a_blocking;  // 获取当前 a 的值 (1)
17         c_blocking = b_blocking;  // 获取当前 b 的值 (1)
18     end
19
20     // 测试非阻塞赋值
21     always @(posedge clk) begin
22         a_nonblocking <= 1;       // 非阻塞赋值: 并行执行
23         b_nonblocking <= a_blocking; // 获取 a 的旧值 (可能是 0)
24         c_nonblocking <= b_blocking; // 获取 b 的旧值 (可能是 0)
25     end
26
27     // 观察输出
28     initial begin
29         #2; // 稍微延迟, 确保仿真开始后捕获信号变化
30         $monitor("Time = %0t | Blocking: a = %b, b = %b, c = %b | NonBlocking: a
31 = %b, b = %b, c = %b",
32                 $time, a_blocking, b_blocking, c_blocking, a_nonblocking,
33                 b_nonblocking, c_nonblocking);
34     end
35 endmodule

```

Fence 28

仿真结果:

1. 在时钟上升沿, `a` 的值立即更新为 1。
2. `b` 的更新被延迟到当前时间步结束, 因此 `b` 会保留上一个 `a` 的值。

时间步	操作	<code>a</code> 值	<code>b</code> 值
t0	初始状态	0	0
t1	时钟上升沿触发	1 (立即更新)	0
t2	<code>b</code> 的值更新完成	1	1

Table 3

阻塞与非阻塞赋值的潜在问题

错误示例：混用阻塞和非阻塞赋值

```

1  module mixed_example(input clk, output reg a, output reg b, output reg c);
2      always @(posedge clk) begin
3          a = b; // 阻塞赋值
4          b <= c; // 非阻塞赋值
5          c = a; // 阻塞赋值

```



```
6 |         end
7 |     endmodule
```

Fence 29

问题分析

- a 的更新依赖于 b 的值，而 b 的更新又依赖于 c。
- 非阻塞赋值的延迟更新与阻塞赋值的立即更新会产生竞态条件，导致结果难以预测。

修正方案

在时钟触发的块中统一使用非阻塞赋值，避免竞态问题。

```
1 | module fixed_example(input clk, output reg a, output reg b, output reg c);
2 |     always @(posedge clk) begin
3 |         a <= b;
4 |         b <= c;
5 |         c <= a;
6 |     end
7 | endmodule
```

Fence 30

阻塞与非阻塞赋值的适用场景

特性	阻塞赋值 (=)	非阻塞赋值 (<=)
执行顺序	按语句顺序逐条执行	并行执行，所有右值同步计算
适用场景	组合逻辑建模	时序逻辑建模
赋值生效时间	当前时间步立即更新	当前时间步结束后更新
推荐使用场景	描述组合逻辑	描述时序逻辑
可能问题	在时序逻辑中可能导致竞态问题	在组合逻辑中使用可能引发非预期延迟
硬件映射	逻辑门 (组合逻辑)	寄存器或触发器 (时序逻辑)

Table 4

综合应用：组合逻辑与时序逻辑的区分

以下代码结合阻塞和非阻塞赋值，展示组合逻辑和时序逻辑的设计差异。

综合示例

```

1  module blocking_nonblocking_example(
2      input clk,
3      input [3:0] a,
4      input [3:0] b,
5      output reg [3:0] sum1,
6      output reg [3:0] sum2
7  );
8      reg [3:0] temp;
9
10     // 阻塞赋值: 组合逻辑
11     always @(*) begin
12         temp = a + b; // 实时计算 temp
13         sum1 = temp;  // 实时更新 sum1
14     end
15
16     // 非阻塞赋值: 时序逻辑
17     always @(posedge clk) begin
18         sum2 <= a + b; // 时钟边沿更新 sum2
19     end
20 endmodule

```

Fence 31

代码分析

1. 阻塞赋值部分：

- 实现组合逻辑，`temp` 和 `sum1` 的值根据输入信号变化实时更新。

2. 非阻塞赋值部分：

- 实现时序逻辑，`sum2` 的值仅在时钟上升沿更新。

小结

1. 阻塞赋值（`=`）：

- 用于组合逻辑描述，按语句顺序逐条执行。
- 不适用于时钟触发逻辑，以避免竞态。

2. 非阻塞赋值（`<=`）：

- 用于时序逻辑描述，赋值延迟到时间步结束后。
- 避免竞态问题，推荐用于时钟触发逻辑。

3. 在设计中：

- 组合逻辑中使用阻塞赋值。
- 时序逻辑中使用非阻塞赋值。

- 避免在同一 `always` 块中混用两种赋值方式。



第三章：模块化设计

模块化设计是 Verilog 的核心特性之一，通过模块实例化可以有效提高代码的复用性和设计的灵活性。通过模块分层，可以将复杂的设计拆解为多个独立的子模块，各模块并行工作，便于设计和验证。

3.1 模块调用与基本设计

模块化设计的作用

1. **分层设计**：将复杂系统分解为多个简单子模块，便于设计和调试。
2. **代码复用**：模块设计可以多次实例化，减少重复劳动。
3. **便于验证**：各模块独立测试后再进行系统集成，降低整体设计难度。

模块实例化的基本方法

设计一个简单的 AND 门模块

```
1 module and_gate(input x, input y, output z);
2     assign z = x & y; // 实现按位 AND 操作
3 endmodule
```

Fence 32

在顶层模块中实例化 AND 门

```
1 module top(input a, input b, output c);
2     and_gate my_and (.x(a), .y(b), .z(c)); // 实例化 and_gate 模块
3 endmodule
```

Fence 33

说明

1. `and_gate` 是子模块，描述 AND 门的逻辑。
2. 在顶层模块 `top` 中，通过 `my_and` 实例化 `and_gate`，实现输入 `a` 和 `b` 的按位与运算。

📁 实际案例：多模块组合

设计一个简单的门控电路，包括 AND 和 OR 门，并在顶层模块中组合它们的逻辑。

子模块设计

1. AND 门子模块：

```
1 module and_gate(input x, input y, output z);
2     assign z = x & y;
3 endmodule
```

Fence 34

2. OR 门子模块：

```
1 module or_gate(input x, input y, output z);
2     assign z = x | y;
3 endmodule
```

Fence 35

顶层模块设计

在顶层模块中组合 AND 和 OR 门，输出的结果为两者逻辑的与：

```
1 module top_logic(input a, input b, output final_out);
2     wire and_out, or_out;
3
4     and_gate u1 (.x(a), .y(b), .z(and_out)); // 实例化 AND 门
5     or_gate u2 (.x(a), .y(b), .z(or_out)); // 实例化 OR 门
6
7     assign final_out = and_out & or_out; // 组合逻辑
8 endmodule
```

Fence 36

说明

- 顶层模块 `top_logic` 实现了输入信号 `a` 和 `b` 的逻辑运算：
 - `and_out` 为 AND 门输出。
 - `or_out` 为 OR 门输出。
 - 最终输出 `final_out` 是两者的按位与结果。

3.2 带参数的模块设计

参数化设计的优势

- **灵活性**：通过参数化调整模块行为，例如位宽、时钟分频系数等。
- **高效性**：编写一次代码，通过参数化适配不同场景。
- **可维护性**：代码结构清晰，便于后续维护和扩展。

参数化模块的定义与实例化

定义带参数的模块

通过 `parameter` 声明参数，使模块的行为更加灵活。

```
1 module param_and_gate #(parameter WIDTH = 4) (  
2     input [WIDTH-1:0] a,  
3     input [WIDTH-1:0] b,  
4     output [WIDTH-1:0] c  
5 );  
6     assign c = a & b; // 参数化位宽的按位 AND 运算  
7 endmodule
```

Fence 37

实例化参数化模块

```
1 module top_with_param(input [7:0] a, input [7:0] b, output [7:0] c);  
2     // 实例化并设置参数 WIDTH 为 8  
3     param_and_gate #(.WIDTH(8)) u1 (  
4         .a(a),  
5         .b(b),  
6         .c(c)  
7     );  
8 endmodule
```

Fence 38

说明

- 在模块 `param_and_gate` 中，参数 `WIDTH` 决定了输入、输出信号的位宽。
- 实例化时通过 `.WIDTH(8)` 设置 `WIDTH` 为 8，使该实例化模块适用于 8 位信号。

实际案例：参数化加法器

参数化模块的定义

设计一个多位加法器，支持动态调整输入信号的位宽。

```
1 module param_adder #(parameter WIDTH = 4) (  
2     input [WIDTH-1:0] a,  
3     input [WIDTH-1:0] b,  
4     output [WIDTH:0] sum // 输出宽度为输入宽度加 1  
5 );  
6     assign sum = a + b; // 参数化加法  
7 endmodule
```

Fence 39

在顶层模块中实例化多个加法器

```
1 module top_with_param_adder(  
2     input [3:0] a, b,  
3     input [7:0] c, d,  
4     output [4:0] sum1, // 4 位加法器输出  
5     output [8:0] sum2 // 8 位加法器输出  
6 );  
7     // 实例化一个 4 位加法器  
8     param_adder #(.WIDTH(4)) adder1 (  
9         .a(a),  
10        .b(b),  
11        .sum(sum1)  
12    );  
13  
14    // 实例化一个 8 位加法器  
15    param_adder #(.WIDTH(8)) adder2 (  
16        .a(c),  
17        .b(d),  
18        .sum(sum2)  
19    );  
20 endmodule
```

Fence 40

功能说明

- adder1 是一个 4 位加法器，计算 $a + b$ 。
- adder2 是一个 8 位加法器，计算 $c + d$ 。

实际应用场景

- 多位加法器广泛用于数据总线计算、地址生成等场景。
 - 参数化设计使加法器能够适配不同位宽的信号，避免重复开发。
-

高级案例：参数化时钟分频器

模块定义

实现一个参数化的时钟分频器，通过参数调整分频倍数。

```
1  module clock_divider #(parameter DIV = 4) (  
2      input clk_in,  
3      output reg clk_out  
4  );  
5      reg [$clog2(DIV)-1:0] counter; // 根据 DIV 设置计数器位宽  
6  
7      always @(posedge clk_in) begin  
8          counter <= counter + 1;  
9          if (counter == DIV/2 - 1) begin  
10             counter <= 0;  
11             clk_out <= ~clk_out; // 翻转输出时钟  
12         end  
13     end  
14 endmodule
```

Fence 41

实例化分频器

在顶层模块中实例化不同倍数的时钟分频器。

```
1  module top_with_dividers(input clk, output clk_2, output clk_4);  
2      // 2 倍分频器  
3      clock_divider #(.DIV(2)) div2 (  
4          .clk_in(clk),  
5          .clk_out(clk_2)  
6      );  
7  
8      // 4 倍分频器  
9      clock_divider #(.DIV(4)) div4 (  
10         .clk_in(clk),  
11         .clk_out(clk_4)  
12     );  
13 endmodule
```

Fence 42

应用场景

- 参数化分频器适用于需要多个不同频率时钟的系统。
 - 可用于时钟同步、电机控制等。
-

小结

1. 模块化设计：

- 将复杂设计分解为多个独立模块，简化开发和验证。
- 通过模块实例化实现并行运行。

2. 参数化设计：

- 通过 `parameter` 提供灵活性，适配不同需求。
- 常见应用包括数据总线处理、时钟分频器等。

3. 实例化参数化模块：

- 提供可复用和可扩展的模块，适用于大规模系统设计。
-

☺ 第四章：函数与任务

Verilog 中的函数（`function`）和任务（`task`）是代码复用的重要工具，用于逻辑封装和模块化设计。它们的功能和硬件实现上有显著差异。在理解 Verilog 函数和任务的同时，我们可以通过 C 语言的类比加深理解。

4.1 函数（Function）

📖 定义与特点

- 函数（`function`）用于封装**纯组合逻辑**，返回单一值，适合描述计算逻辑。
- 特点：
 - 1. **必须有返回值**，结果可用于赋值或表达式。
 - 2. **只能包含组合逻辑**，不支持时序控制语句（如 `@` 和 `#`）。
 - 3. 函数的变量是局部的，不能直接操作外部信号。
- 适用场景：
 - 封装简单的数学运算（如加法、乘法、比较）。
 - 用于需要单一结果返回的场景。

📖 Verilog 函数示例

示例 1：4 位加法器

功能：实现两个 4 位输入信号的加法，返回结果。

```
1  function [4:0] add;
2      input [3:0] a, b;
3      begin
4          add = a + b; // 返回加法结果，输出宽度为 5 位
5      end
6  endfunction
7
8  module function_example(input [3:0] a, b, output [4:0] sum);
9      assign sum = add(a, b); // 调用函数
10 endmodule
```

Fence 43

示例 2：最大值比较器

功能：返回两个 4 位数中的较大值。

```
1  function [3:0] max;  
2      input [3:0] a, b;  
3      begin  
4          if (a > b)  
5              max = a;  
6          else  
7              max = b;  
8          end  
9      endfunction  
10  
11 module max_example(input [3:0] a, b, output [3:0] max_val);  
12     assign max_val = max(a, b); // 调用函数  
13 endmodule
```

Fence 44

📦 C 语言对比：函数实现

示例 1：加法函数

功能：返回两个整数的和。

```
1  #include <stdio.h>  
2  
3  // 函数定义  
4  int add(int a, int b) {  
5      return a + b; // 返回加法结果  
6  }  
7  
8  int main() {  
9      int a = 5, b = 3;  
10     int sum = add(a, b); // 调用函数  
11     printf("Sum: %d\n", sum); // 输出结果  
12     return 0;  
13 }
```

Fence 45

示例 2：最大值比较器

功能：返回两个整数中的较大值。

```
1  #include <stdio.h>  
2  
3  // 函数定义  
4  int max(int a, int b) {  
5      if (a > b)  
6          return a;  
7      else  
8          return b;
```

```
9   }
10
11   int main() {
12       int a = 7, b = 9;
13       int result = max(a, b); // 调用函数
14       printf("Max: %d\n", result); // 输出最大值
15       return 0;
16   }
```

Fence 46

4.2 任务 (Task)

📖 定义与特点

○ **任务 (task)** 是一种过程描述工具，可以封装复杂逻辑，包括组合逻辑和时序逻辑。

○ **特点：**

1. **无返回值**，通过操作外部变量实现输出。
2. **支持时序控制语句**，例如 `@` 和 `#`。
3. 可操作多个信号，适合复杂逻辑描述。

○ **适用场景：**

- 描述复杂逻辑，如计数器、状态机。
- 封装时序逻辑，包括延迟操作和条件控制。

📖 Verilog 任务示例

示例 1：条件计数器

功能：在时钟触发下，计数器在使能信号为高时递增。

```
1   task conditional_counter_task;
2       input clk, enable;
3       output reg [3:0] counter;
4       begin
5           if (enable) begin
6               counter = counter + 1; // 条件递增
7           end
8       end
9   endtask
10
11   module task_example(input clk, enable, output reg [3:0] counter);
12       always @(posedge clk) begin
13           conditional_counter_task(clk, enable, counter); // 调用任务
14       end
```

```
15 | endmodule
```

Fence 47

示例 2：状态机任务

功能：封装状态跳转逻辑。

```
1 | task state_transition_task;
2 |     input [1:0] state;
3 |     output reg [1:0] next_state;
4 |     begin
5 |         case (state)
6 |             2'b00: next_state = 2'b01;
7 |             2'b01: next_state = 2'b10;
8 |             2'b10: next_state = 2'b00;
9 |             default: next_state = 2'b00;
10 |        endcase
11 |    end
12 | endtask
13 |
14 | module state_machine(input [1:0] state, output reg [1:0] next_state);
15 |     always @(*) begin
16 |         state_transition_task(state, next_state); // 调用任务
17 |     end
18 | endmodule
```

Fence 48

🔗 C 语言对比：任务实现

示例 1：条件计数器

功能：仅当使能信号为高时递增计数器。

```
1 | #include <stdio.h>
2 |
3 | // 条件计数器任务
4 | void conditional_counter_task(int enable, int *counter) {
5 |     if (enable) {
6 |         (*counter)++; // 条件递增
7 |     }
8 | }
9 |
10 | int main() {
11 |     int counter = 0;
12 |     int enable = 1;
13 |
14 |     // 调用任务
15 |     conditional_counter_task(enable, &counter);
16 |     printf("Counter: %d\n", counter);
17 |     return 0;
18 | }
```

Fence 49

示例 2：状态机任务

功能：实现状态机的状态跳转。

```
1  #include <stdio.h>
2
3  // 状态机任务
4  void state_transition_task(int state, int *next_state) {
5      switch (state) {
6          case 0: *next_state = 1; break;
7          case 1: *next_state = 2; break;
8          case 2: *next_state = 0; break;
9          default: *next_state = 0; break;
10     }
11 }
12
13 int main() {
14     int state = 0, next_state;
15
16     // 调用任务
17     state_transition_task(state, &next_state);
18     printf("Next State: %d\n", next_state);
19     return 0;
20 }
```

Fence 50

4.3 函数与任务的区别

特性	函数（Function）	任务（Task）
返回值	必须有返回值	无返回值，通过外部变量传递结果
操作范围	仅限于局部信号	可操作局部和外部信号
支持时序逻辑	不支持	支持（如 @ 和 # ）
操作信号数量	通常只操作单个信号	可同时操作多个信号
调用方式	嵌入表达式调用	必须作为独立语句调用
硬件映射	仅描述组合逻辑	可描述组合和时序逻辑

Table 5

4.4 小结

1.函数（Function）：

- 用于封装轻量级的组合逻辑运算，必须有返回值。
- 适用于加法器、比较器等单一运算逻辑。

2.任务（Task）：

- 用于封装复杂的逻辑操作，支持组合逻辑和时序逻辑。
- 适用于计数器、状态机等过程性操作。

3. c 类比：

- Verilog 函数类似于 C 中带返回值的函数。
 - Verilog 任务类似于 C 中 `void` 函数，通过参数修改变量值。
-

第五章：有限状态机（FSM）

5.1 FSM 基础

定义

有限状态机（FSM, Finite State Machine）是一种数学模型，用于描述一个系统在有限数量的状态之间的切换，状态的变化由输入信号触发。FSM 广泛应用于数字电路设计、控制系统以及通信协议中，常用于建模复杂的时序逻辑电路和离散事件系统。

FSM 的组成部分

- 状态集合**：系统中可能的所有状态。
- 输入集合**：驱动状态机转换的输入信号。
- 输出集合**：由状态或状态与输入信号决定的输出。
- 状态转移条件**：定义了状态之间的转换规则。
- 初始状态**：系统启动时的默认状态。

FSM 的分类

FSM 可分为以下两种主要类型：

特性	Moore 状态机	Mealy 状态机
输出依赖	仅依赖当前状态	依赖当前状态和输入
输出变化时机	状态发生变化后	输入变化时，输出可能立即更新
硬件实现	更容易综合，输出逻辑简单	可能需要更复杂的组合逻辑
应用场景	状态驱动的设计（如交通灯、流水灯）	输入变化敏感的设计（如协议解析器）

Table 6

5.2 FSM 的 Verilog 实现

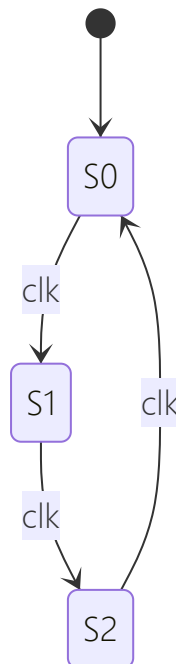
Moore 状态机实现

示例：简单的状态循环 (S0 -> S1 -> S2 -> S0)

```
1 module fsm_moore(input clk, input rst, output reg [1:0] state);
2     parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
3
4     // 状态转移逻辑
5     always @(posedge clk or posedge rst) begin
6         if (rst)
7             state <= S0; // 复位到初始状态
8         else
9             case (state)
10                S0: state <= S1;
11                S1: state <= S2;
12                S2: state <= S0;
13            endcase
14        end
15    endmodule
```

Fence 51

对应状态图



Mealy 状态机实现

示例：根据输入决定状态和输出

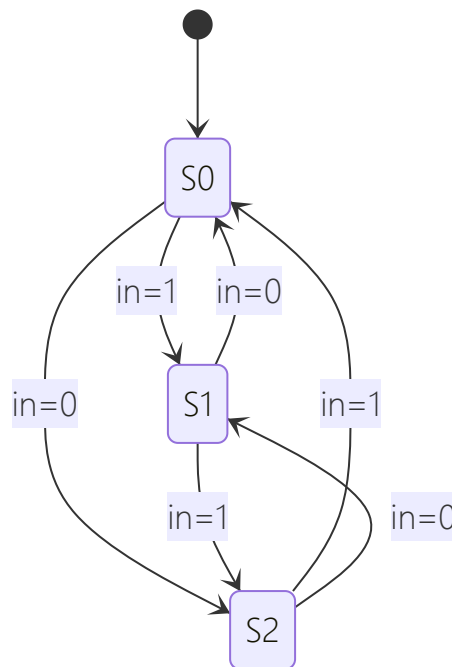
```

1  module fsm_mealy(input clk, input rst, input in, output reg out, output reg [1:0]
   state);
2      parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
3
4      // 状态转移逻辑
5      always @(posedge clk or posedge rst) begin
6          if (rst)
7              state <= S0; // 复位到初始状态
8          else
9              case (state)
10                 S0: if (in) state <= S1; else state <= S2;
11                 S1: if (in) state <= S2; else state <= S0;
12                 S2: if (in) state <= S0; else state <= S1;
13             endcase
14          end
15
16      // 输出逻辑
17      always @(*) begin
18          case (state)
19             S0: out = in ^ 1'b0;
20             S1: out = in ^ 1'b1;
21             S2: out = in ^ 1'b0;
22          endcase
23      end
24  endmodule

```

Fence 52

对应状态图



5.3 FSM 的实际应用示例

案例一：交通灯控制系统

设计目标

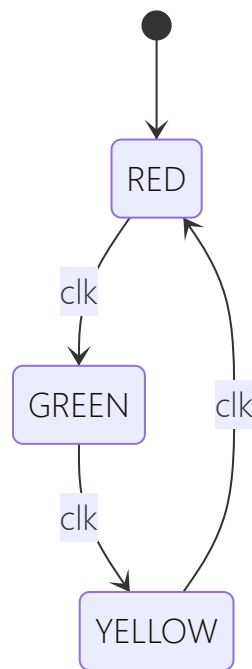
1. 使用 Moore 状态机实现简单的交通灯控制。
2. 三个状态：
 - 红灯 (RED) 。
 - 绿灯 (GREEN) 。
 - 黄灯 (YELLOW) 。
3. 每个状态持续一定的时钟周期。

Verilog 实现

```
1  module traffic_light_fsm(  
2      input clk,  
3      input rst,  
4      output reg [1:0] light  
5  );  
6      parameter RED = 2'b00, GREEN = 2'b01, YELLOW = 2'b10;  
7      reg [1:0] state, next_state;  
8  
9      // 状态转移逻辑  
10     always @(posedge clk or posedge rst) begin  
11         if (rst)  
12             state <= RED;  
13         else  
14             state <= next_state;  
15     end  
16  
17     // 下一个状态逻辑  
18     always @(*) begin  
19         case (state)  
20             RED: next_state = GREEN;  
21             GREEN: next_state = YELLOW;  
22             YELLOW: next_state = RED;  
23             default: next_state = RED;  
24         endcase  
25     end  
26  
27     // 输出逻辑  
28     always @(*) begin  
29         light = state; // 状态直接决定输出 (Moore)  
30     end  
31 endmodule
```

Fence 53

对应状态图



案例二：密码锁设计

设计目标

- 1.使用 Mealy 状态机实现简单的密码锁。
- 2.假设密码为“110”，当用户输入正确序列时，解锁。

Verilog 实现

```

1  module password_lock(
2      input clk,
3      input rst,
4      input in,
5      output reg unlock
6  );
7      parameter IDLE = 2'b00, S1 = 2'b01, S2 = 2'b10, UNLOCK = 2'b11;
8      reg [1:0] state, next_state;
9
10     // 状态转移逻辑
11     always @(posedge clk or posedge rst) begin
12         if (rst)
13             state <= IDLE;
14         else
15             state <= next_state;
16     end
17
18     // 下一个状态逻辑
19     always @(*) begin
20         case (state)

```

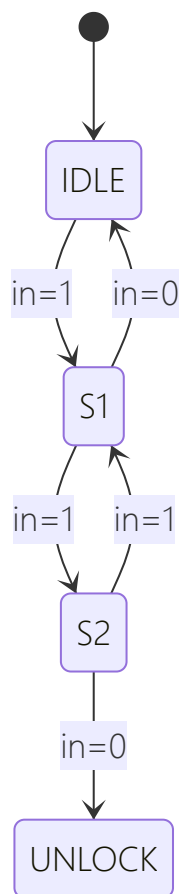
```

21         IDLE: next_state = (in) ? S1 : IDLE;
22         S1: next_state = (in) ? S2 : IDLE;
23         S2: next_state = (in) ? UNLOCK : IDLE;
24         UNLOCK: next_state = UNLOCK;
25         default: next_state = IDLE;
26     endcase
27 end
28
29 // 输出逻辑
30 always @(*) begin
31     unlock = (state == UNLOCK);
32 end
33 endmodule

```

Fence 54

对应状态图



5.4 小结

1. FSM 的分类：

- **Moore 状态机**：输出仅依赖当前状态，设计简单，适用于状态驱动型应用。
- **Mealy 状态机**：输出依赖于当前状态和输入，适用于输入敏感型应用。

2. 实际应用场景：

- Moore 状态机：交通灯控制、电梯状态管理。
 - Mealy 状态机：通信协议解析、密码锁控制。
-